Report

Of

Cybersecurity Internship: Phishing Detection System

Submitted

To

School of Computer Science and Engineering IILM University,

Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE



By

Roll Number of Student: CS-2341642

Name of Student: AMAN KUMAR GAUTAM

Batch: 2023-27

Declaration

I, **Aman Kumar Gautam**, Roll Number **CS-2341642**, a student of B.Tech Computer Science and Engineering (Batch 2023-27) at the School of Computer Science and Engineering, IILM University, Greater Noida, U.P., hereby declare that this internship report, titled **"Phishing Detection System"**, is a record of the work carried out by me during my Cybersecurity internship.

The work described in this report is original and was carried out by me. All sources of information, tools and libraries that I have used are acknowledged in the text and in the list of references. This report has not been submitted, in part or in full, to any other university or institution for the award of any degree or diploma.

Date: 06 October 2026

Aman Kumar Gautam
Roll No.: CS-2341642

–

Acknowledgement

I would like to express my sincere gratitude to everyone who supported me during this internship and the preparation of this report.

I am thankful to my internship mentors for assigning me a meaningful and practical problem in the field of cybersecurity, and for their guidance, feedback and encouragement throughout the work. The task of building a machine learning model to detect phishing emails helped me connect classroom concepts with a real security challenge.

I also thank the faculty members of the School of Computer Science and Engineering, IILM University, Greater Noida, for the strong foundation in programming, data and machine learning that made this work possible, and for allowing internships to be a part of the B.Tech curriculum.

Finally, I am grateful to my family and friends for their constant support and motivation.

Abstract

Phishing is one of the most common ways in which attackers steal credentials and money. A phishing email imitates a trusted sender and pushes the reader to click a malicious link or share sensitive information, usually by creating a sense of urgency. Because such emails are sent in very large numbers and are written to look genuine, checking them by hand does not scale.

This report describes the design and implementation of a **Phishing Detection System** built with Python and Scikit-learn as part of a cybersecurity internship. The model reads the subject and body of an email and classifies it as **Phishing** or **Safe**, together with a confidence score. Two kinds of features are used: TF-IDF features that capture which words and word pairs are characteristic of each class, and six engineered “red-flag” features — URL count, suspicious-keyword count, exclamation marks, fully capitalised words, digit ratio and text length. The two feature sets are merged into one matrix and used to train a Random Forest classifier of 300 trees with balanced class weights.

On a labelled dataset of 360 emails (180 phishing and 180 safe), split 75% / 25% for training and testing, the model classified all 90 test emails correctly and is evaluated with accuracy, a confusion matrix and a precision/recall report. The report is open about the limits of this result: the dataset is synthetic and template-based, so the perfect score validates the pipeline rather than proving real-world performance. In addition to the core model, the project provides a script to check any pasted email, a script to merge a real Kaggle dataset with the synthetic one, and an IMAP-based script that scans a live inbox automatically.

Keywords: phishing detection, machine learning, Scikit-learn, TF-IDF, Random Forest, cybersecurity, text classification.

Contents

Declaration **2**

Acknowledgement **3**

Abstract **4**

1. Introduction **7**

1.1 Background 7

1.2 Problem Statement 7

1.3 Objectives 7

1.4 Scope 8

1.5 Organisation of the Report 8

2. Internship Overview **9**

2.1 Domain 9

2.2 Assigned Task and How It Was Addressed 9

2.3 Work Carried Out 9

3. Tools and Technologies **10**

3.1 Software and Libraries 10

3.2 Development Environment 10

3.3 Key Concepts 10

4. Methodology and System Design **11**

4.1 Architecture Overview 11

4.2 Dataset 11

4.3 Feature Extraction 12

4.4 Model and Training Setup 12

4.5 Evaluation Metrics 13

4.6 Classifying a New Email 13

5. Implementation **14**

5.1 Project Structure 14
5.2 Core Module 14
5.3 Checking a Single Email 14
5.4 Merging a Real Dataset 15
5.5 Automatic Inbox Scanning 15
5.6 Running the Project 15

6. Results and Analysis **16**

6.1 Experimental Setup 16
6.2 Performance on the Test Set 16
6.3 Most Important Features 17
6.4 Sample Predictions 18
6.5 Discussion 18

7. Challenges and Solutions **19**

8. Learning Outcomes **20**

8.1 Technical Skills 20
8.2 Cybersecurity Understanding 20
8.3 Professional Skills 20

9. Limitations and Future Scope **21**

9.1 Limitations 21
9.2 Future Scope 21

10. Conclusion **22**

References **23**

Appendix A: Running the Project **24**

1. Introduction

1.1 Background

Email is still one of the most widely used channels for communication in companies, universities and everyday life. That also makes it one of the most attractive channels for attackers. **Phishing** is a social-engineering attack in which the attacker sends a message that pretends to come from a trusted party — a bank, an online store, a delivery service or even a colleague — in order to trick the recipient into revealing a password, making a payment or opening a malicious link.

Phishing emails tend to rely on the same few psychological levers: **urgency** ("your account will be suspended in 24 hours"), **fear**, **authority** and the promise of a **reward**. They usually contain a link to a look-alike website, and they often use a generic greeting such as "Dear Customer". Since the messages are sent in bulk and are written to look genuine at a glance, relying only on people to spot them does not scale.

Machine learning offers a practical way to support human judgement. Instead of relying on a fixed blacklist, a model learns the statistical patterns of language and structure from labelled examples of phishing and legitimate emails, and can then score new emails it has never seen.

1.2 Problem Statement

The task assigned during the internship was to build a machine learning model using Scikit-learn that can train on a dataset of phishing and legitimate emails, extract and analyse email features such as URLs and keywords, classify emails as "Phishing" or "Safe", and display the accuracy and the confusion matrix. The expected outcome was that the model classifies emails as Phishing or Safe with high accuracy based on their textual content.

1.3 Objectives

- To train a classifier on a dataset of phishing and legitimate emails.
- To extract and analyse email features such as URLs, suspicious keywords and word patterns.
- To classify any new email as "Phishing" or "Safe" and report how confident the model is.
- To evaluate the model using accuracy, a confusion matrix and precision/recall.
- To make the model practical to use: check a single pasted email, merge a real dataset, and scan a live inbox.

1.4 Scope

The model works on the **text** of an email (subject and body). It does not inspect attachments, images, sender authentication records (SPF, DKIM, DMARC) or the reputation of the linked websites. The project is an academic prototype intended to demonstrate the complete machine learning workflow on a cybersecurity problem.

1.5 Organisation of the Report

Chapter 2 gives an overview of the internship task and the work carried out. Chapter 3 lists the tools and concepts used. Chapter 4 explains the methodology and system design, and Chapter 5 describes the implementation. Chapter 6 presents and analyses the results. Chapters 7 to 10 cover the challenges faced, the learning outcomes, the limitations with future scope, and the conclusion.

2. Internship Overview

2.1 Domain

The internship was in the domain of **cybersecurity**, with the project applying machine learning to the problem of phishing detection. The work combined data preparation, feature engineering, model training and evaluation, and a few practical tools built around the trained model.

2.2 Assigned Task and How It Was Addressed

Requirement	How it was addressed
Train on a dataset of phishing and legitimate emails	A labelled dataset of 360 emails (180 phishing, 180 safe) is generated by `generate_dataset.py`; a real Kaggle dataset can be added with `merge_datasets.py`.
Extract and analyse email features (URLs, keywords, etc.)	TF-IDF word and word-pair features, plus six engineered features: URL count, suspicious-keyword count, exclamation marks, capitalised words, digit ratio and text length.
Classify emails as “Phishing” or “Safe”	A Random Forest classifier; `predict_email()` returns the label with class probabilities.
Display accuracy and confusion matrix	Accuracy, confusion matrix and a classification report are printed, and the matrix is saved as `confusion_matrix.png`.

Table 2.1: Task requirements and their implementation

2.3 Work Carried Out

Phase	Activity	Output
1. Problem study	Studied the common indicators of phishing: urgency, links, credential requests, look-alike domains.	List of features to extract
2. Dataset preparation	Created a labelled dataset of phishing and legitimate emails.	data/emails_dataset.csv
3. Feature engineering	Built TF-IDF features and six red-flag features; combined them into one matrix.	Feature matrix
4. Training and evaluation	Trained a Random Forest; measured accuracy, confusion matrix, precision and recall.	Metrics and confusion_matrix.png
5. Practical extensions	Wrote tools to check a pasted email, merge a Kaggle dataset and scan an inbox.	Three additional scripts
6. Documentation	Wrote the README and published the project source code on GitHub.	README and repository

Table 2.2: Phases of the work

3. Tools and Technologies

3.1 Software and Libraries

Tool / Library	Purpose in the project
Python 3	Programming language used for the whole project.
Pandas	Loads the CSV dataset and handles the subject, body and label columns.
Scikit-learn	TfidfVectorizer, RandomForestClassifier, train_test_split and the evaluation metrics.
SciPy	`hstack` and `csr_matrix` to combine sparse TF-IDF features with dense engineered features.
NumPy	Ranks the features by importance (`argsort`).
Matplotlib	Draws and saves the confusion matrix image.
re, csv, random (built-in)	URL detection with regular expressions, reading and writing the dataset, random generation of sample emails.
imaplib, email (built-in)	Connect to a mailbox over IMAP and parse the subject and body of each message.

Table 3.1: Tools and libraries

3.2 Development Environment

The project was developed in **Visual Studio Code** using a Python **virtual environment (venv)** so that the libraries installed for this project stay separate from other projects. Packages were installed with `pip`, and the source code was hosted on **GitHub** for version control and sharing.

3.3 Key Concepts

- **Supervised learning** — the model learns from examples that already carry the correct label (phishing or safe) and then predicts the label of new examples.
- **Text classification** — assigning a category to a piece of text; here, the category of an email.
- **TF-IDF** — a way of turning text into numbers by weighting each word by how often it appears in an email and how rare it is across all emails, so that distinctive words count for more than common ones.
- **Feature engineering** — designing extra input signals (such as the number of URLs) from domain knowledge.
- **Random Forest** — an ensemble of many decision trees that vote on the answer.
- **Confusion matrix** — a table that shows how many emails of each true class were predicted as each class.

4. Methodology and System Design

4.1 Architecture Overview

The system is a pipeline: data moves from one stage to the next until a prediction is produced. A raw email (subject and body) is converted into numeric features, the Random Forest model scores those features, and the result is returned as a label with a confidence for each class (Figure 4.1).

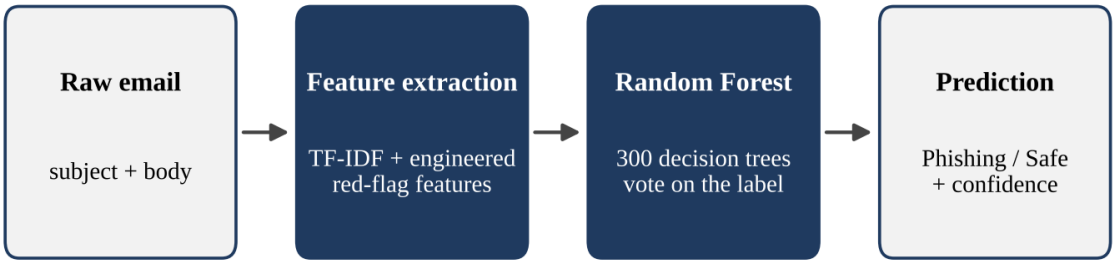


Figure 4.1: Pipeline of the phishing detection system

4.2 Dataset

The dataset is a CSV file with three columns: `subject`, `body` and `label`. It contains 360 emails, balanced between the two classes. The emails were generated from templates so that the whole pipeline could be built and tested without first collecting and cleaning a large real corpus.

The phishing emails are built from 10 subject lines, 10 body templates and 8 suspicious links (including look-alike domains such as a misspelt brand name and a link that uses a raw IP address). The legitimate emails are built from 10 subject lines and 15 everyday bodies such as meeting notes, invoices, appointment reminders, shipping notices and flight confirmations. The rows are shuffled with a fixed seed (42) for repeatability.

Class	Rows	Unique texts
Phishing	180	164
Safe	180	106
Total	360	270

Table 4.1: Composition of the synthetic dataset

Because the emails come from a limited set of templates, some rows are exact duplicates; the effect of this on the results is discussed in Section 6.5. A real dataset can be used by replacing or merging the CSV file.

4.3 Feature Extraction

4.3.1 TF-IDF text features

The subject and body are joined into one string and converted with Scikit-learn's `TfidfVectorizer`, using English stop-word removal, single words and word pairs (unigrams and bigrams), and a limit of 1,500 features. For a term *t* in an email *d*, TF-IDF = tf(*t*, *d*) × idf(*t*), where idf(*t*) = ln((1 + N) / (1 + df(*t*))) + 1, N is the number of emails and df(*t*) is the number of emails containing the term. Words that are common to every email receive a low weight, while distinctive words such as "verify" or "urgent" receive a high weight.

4.3.2 Engineered red-flag features

TF-IDF captures *which words* are used but not the structure of the message. Six hand-crafted features add that information:

Feature	Description
url_count	Number of links found with the regular expression `https?://\S+|www\.\S+`.
suspicious_keyword_count	Total occurrences of 22 suspicious words and phrases, for example urgent, verify, suspend, immediately, click here, confirm, password, account, act now, winner, claim, security alert, unusual activity, final notice, login, credentials and bank.
exclamation_count	Number of exclamation marks, a common sign of pressure.
uppercase_word_count	Number of words written fully in capitals (longer than one letter), such as URGENT.
digit_ratio	Share of characters that are digits.
text_length	Total number of characters in the email.

Table 4.2: Engineered features

4.3.3 Combining the two feature sets

The TF-IDF output is a sparse matrix (mostly zeros), while the engineered features are a small dense table. They are merged column-wise with SciPy's `hstack` after converting the dense part with `csr_matrix`. The result is a single matrix that the classifier can use, with the TF-IDF columns followed by the six engineered columns.

4.4 Model and Training Setup

A **Random Forest** classifier was chosen. It builds many decision trees, each trained on a different random sample of the data and features, and the final label is decided by majority vote. This reduces the over-fitting that a single decision tree is prone to, works well with a mix of sparse and numeric features, needs little tuning, and provides feature importances that help explain the model's decisions.

Setting	Value
Number of trees (n_estimators)	300
Maximum depth	None (trees grow until pure)
Class weights	balanced
Random seed (random_state)	42
Train / test split	75% / 25%, stratified by label (270 training and 90 test emails)

Table 4.3: Model and training configuration

4.5 Evaluation Metrics

Treating *phishing* as the positive class, the confusion matrix has four cells: true positives (TP: phishing correctly flagged), false negatives (FN: phishing missed), false positives (FP: safe email wrongly flagged) and true negatives (TN: safe email correctly passed).

Metric	Formula	Meaning
Accuracy	(TP + TN) / total	Share of all emails classified correctly.
Precision	TP / (TP + FP)	When the model says phishing, how often it is right.
Recall	TP / (TP + FN)	Of all real phishing emails, how many were caught.
F1-score	2 × P × R / (P + R)	Balance between precision and recall.

Table 4.4: Evaluation metrics

In a security setting a false negative (a phishing email passed as safe) is usually the more dangerous error, so recall on the phishing class deserves particular attention alongside overall accuracy.

4.6 Classifying a New Email

For a new email, the saved vectoriser and the engineered-feature function are applied to its text, the two feature sets are combined in the same way as in training, and the trained forest returns both the predicted label and the probability of each class. The probabilities act as the confidence score.

5. Implementation

5.1 Project Structure

File	Role
generate_dataset.py	Generates the labelled synthetic dataset (data/emails_dataset.csv).
phishing_detector.py	Core module: feature extraction, training, evaluation, confusion matrix and predict_email().
check_my_email.py	Classifies an email that the user pastes into the script.
merge_datasets.py	Converts a Kaggle dataset to the project format and merges it with the synthetic one.
check_inbox.py	Fetches recent emails from a mailbox over IMAP and classifies each one.
README.md	Short description of the project and how to run it.
confusion_matrix.png	Saved confusion matrix of the latest run.

Table 5.1: Files in the project

5.2 Core Module

The main steps of `phishing_detector.py` are the vectorisation of the training and test text, the merging of the TF-IDF and engineered features, and the training of the forest:

```
vectorizer = TfidfVectorizer(max_features=1500, stop_words="english",
                             ngram_range=(1, 2))
X_train_tfidf = vectorizer.fit_transform(X_train_text)
X_test_tfidf  = vectorizer.transform(X_test_text)

X_train = hstack([X_train_tfidf, csr_matrix(train_eng.values)])
X_test  = hstack([X_test_tfidf,  csr_matrix(test_eng.values)])

clf = RandomForestClassifier(n_estimators=300, random_state=42,
                             class_weight="balanced")
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

The vectoriser is fitted on the training emails only and then applied to the test emails, so no information from the test set leaks into the feature vocabulary. After prediction the script prints the accuracy, the confusion matrix and the classification report, saves the confusion matrix as an image, and lists the most important features.

5.3 Checking a Single Email

`check_my_email.py` lets a user test any email without writing code. The user pastes the subject and body into one variable, runs the script, and receives a verdict (Phishing or Safe) with the confidence for each class. This makes the model easy to demonstrate to people who are not programmers.

5.4 Merging a Real Dataset

A model trained only on template emails cannot be trusted on real mail, so `merge_datasets.py` was written to bring in a real dataset from Kaggle. Different Kaggle datasets use different column names and label formats (for example text labels such as “Phishing Email”, or 0/1 codes). The script therefore has a small settings block where the text column, the label column and a label-mapping are set. It then renames the columns to a common format, maps the labels to “phishing” and “safe”, drops rows with missing values, combines both datasets, removes exact duplicates and saves the result as `data/emails_dataset_merged.csv`. The loader in the core module accepts both the original three-column file and the merged file.

5.5 Automatic Inbox Scanning

`check_inbox.py` connects to a mailbox over **IMAP** using Python’s `imaplib`, fetches the most recent emails (five by default), decodes the subject, extracts the plain-text body and classifies each message with the trained model. For every email it prints the sender, the subject, the verdict and the class probabilities.

Security was considered in the design. The script is meant to be used with an **app password** generated by the email provider (which needs two-step verification), never with the real account password, and it can read the address and app password from environment variables so that credentials need not be written into the file or uploaded to GitHub.

5.6 Running the Project

The scripts are run in order: first the dataset generator, then the core module. The exact commands are listed in Appendix A. Working inside a dedicated virtual environment keeps the installed libraries isolated from the other projects on the same computer.

6. Results and Analysis

6.1 Experimental Setup

Item	Value
Dataset size	360 emails (180 phishing, 180 safe)
Training set	270 emails
Test set	90 emails (45 phishing, 45 safe)
Features	TF-IDF (unigrams and bigrams, up to 1,500) + 6 engineered features
Classifier	Random Forest, 300 trees, balanced class weights

Table 6.1: Experimental setup

6.2 Performance on the Test Set

The model classified all 90 test emails correctly, giving an **accuracy of 100%**. Both the precision and the recall are 1.00 for each class, so there were no false positives and no false negatives (Figure 6.1 and Tables 6.2 and 6.3).

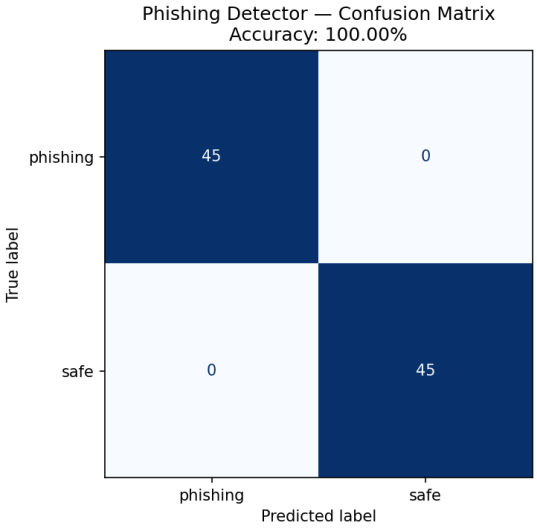


Figure 6.1: Confusion matrix on the 90 test emails

Actual \ Predicted	Phishing	Safe
Phishing	45	0
Safe	0	45

Table 6.2: Confusion matrix

Class	Precision	Recall	F1-score	Support
Phishing	1.00	1.00	1.00	45
Safe	1.00	1.00	1.00	45
Accuracy			1.00	90

Table 6.3: Classification report

6.3 Most Important Features

Figure 6.2 shows the ten features the forest relied on most. The top two are engineered features: **url_count** (0.1310) and **suspicious_keyword_count** (0.1060). They are followed by the token “http”, the text length, and words such as “account”, “verify”, “login” and “immediately”. These are the signals one would expect from phishing: links, credential-related words and urgency. This also shows that adding the engineered features was worthwhile, since they outrank every individual word.

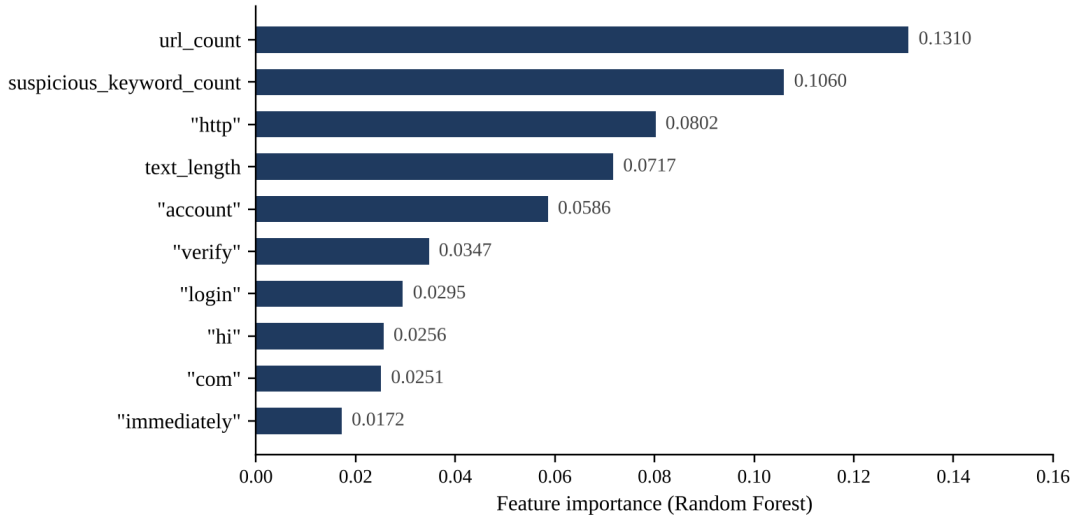


Figure 6.2: Ten most important features

Some entries reflect the synthetic data rather than genuine phishing behaviour. For example, “hi” is important because several of the safe templates begin with a friendly greeting, and text length is useful because the templates differ in length. Such features would not be expected to carry over to real emails.

6.4 Sample Predictions

Two new emails that were not in the dataset were classified after training:

Email text	Prediction	P(phishing)	P(safe)
URGENT: Your account will be suspended! Click http://verify-now-secure.com immediately to confirm your password and avoid permanent closure.	Phishing	0.9133	0.0867
Hi Sarah, attached is the agenda for tomorrow’s 10am standup. Let me know if you’d like to add anything.	Safe	0.0000	1.0000

Table 6.4: Predictions on unseen emails

6.5 Discussion

A score of 100% should be read with care. It shows that the pipeline works from end to end, but it does not show that the model would be this accurate on real phishing emails, for two reasons:

- **The data is template-based.** Phishing and safe emails come from very different sets of templates, which makes the two classes easy to separate.
- **The data contains duplicates.** Only 270 of the 360 rows are unique, and 33 of the 90 test emails also appear, word for word, in the training set. The model is therefore partly being tested on emails it has already seen.

For these reasons the result is best seen as a successful validation of the pipeline. A fair estimate of real-world performance requires a real dataset, such as the one that can be merged using the provided script, with duplicates removed before the train/test split.

7. Challenges and Solutions

Challenge	How it was handled
Combining very different feature types: thousands of sparse TF-IDF columns and a few dense numeric features.	Converted the dense features with csr_matrix and merged both with SciPy’s hstack into one matrix.
Python environment confusion in VS Code: a virtual environment was created in the wrong folder because the workspace held several projects and the project folder was not the root.	Opened the project folder itself as the VS Code root, created the venv there, and installed the libraries inside it.
A perfect score looked too good to be true.	Investigated the dataset, found that it is template-based with duplicate rows, and documented it as a limitation instead of presenting 100% as real-world accuracy.
Real datasets have different column names and label formats.	Wrote a merge script with a configurable settings block that maps any dataset to the common format.
Reading a live mailbox without exposing a password.	Used IMAP with an app password and environment variables, and kept credentials out of the repository.

Table 7.1: Challenges faced and solutions

8. Learning Outcomes

8.1 Technical Skills

- Completing the full machine learning workflow: data, features, training, evaluation and prediction.
- Representing text numerically with TF-IDF, and extending it with domain-specific features.
- Training and interpreting a Random Forest, including class weights and feature importances.
- Reading a confusion matrix and the precision, recall and F1 metrics, and choosing which error matters most.
- Working with Pandas, SciPy sparse matrices, regular expressions and Python’s email libraries.

8.2 Cybersecurity Understanding

- Phishing succeeds by combining several weak signals — urgency, links, generic greetings and credential requests — rather than by any single keyword.
- Good security tooling must be honest about its limits; a high score on easy data is not proof of protection.
- Handling credentials safely: app passwords, environment variables and keeping secrets out of public code.

8.3 Professional Skills

- Setting up and managing a development environment with VS Code and virtual environments.
- Documenting a project clearly and publishing it on GitHub.
- Explaining technical work to different audiences, from a short summary to a detailed report.

9. Limitations and Future Scope

9.1 Limitations

- The training data is small, synthetic and contains duplicates, so the reported accuracy does not represent real-world performance.
- Only the text of the email is analysed; attachments, images and embedded content are ignored.
- Sender authentication (SPF, DKIM, DMARC), the sender's domain and the reputation of linked websites are not checked.
- An attacker who writes a message without the usual keywords may avoid detection.
- The inbox scanner has only been designed for light personal use; it is not a production mail filter.

9.2 Future Scope

- Train and test on a large real dataset, removing duplicates before the split and using cross-validation for a more reliable estimate.
- Add sender-based features such as domain mismatch, display-name spoofing and authentication results, and analyse the URLs themselves (length, IP addresses, look-alike domains).
- Compare other models, such as Logistic Regression, linear SVM and gradient boosting, and tune hyper-parameters with GridSearchCV.
- Save the trained model with joblib so it can be loaded without retraining.
- Deploy it as a browser extension, an email plugin or a small web service for real-time checking.

10. Conclusion

This internship project delivered a working machine learning model that classifies emails as Phishing or Safe from their text. It combines TF-IDF features with six engineered red-flag features, uses a 300-tree Random Forest, and reports accuracy, a confusion matrix and precision/recall. On the test set of 90 emails, the model classified every email correctly, and on two unseen sample emails it produced the expected labels with sensible confidence scores.

Just as important is what the project showed about evaluation. The perfect score comes from a synthetic, template-based dataset that contains duplicate rows, so it confirms that the pipeline is correct rather than demonstrating real-world accuracy. The additional tools — a single-email checker, a dataset merger for real Kaggle data and an IMAP inbox scanner — make the next step, validation on real emails, straightforward.

Overall, the internship gave me hands-on experience of the complete machine learning workflow on a genuine cybersecurity problem, together with a clearer understanding of how phishing works and why careful, honest evaluation matters in security.

References

[1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[2] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[3] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[4] M. Crispin, “Internet Message Access Protocol – Version 4rev1,” RFC 3501, Internet Engineering Task Force, 2003.

[5] Scikit-learn documentation, “TfidfVectorizer”, “RandomForestClassifier” and “Model evaluation”. https://scikit-learn.org/stable/

[6] Python Software Foundation, “imaplib – IMAP4 protocol client” and “email – An email and MIME handling package”. https://docs.python.org/3/

[7] Anti-Phishing Working Group (APWG), information on phishing and e-mail fraud. https://apwg.org

[8] Kaggle, public phishing email datasets. https://www.kaggle.com/datasets

Appendix A: Running the Project

1. Install the libraries (inside the project's virtual environment):

```
pip install pandas scikit-learn matplotlib scipy
```

2. Create the dataset (once):

```
python generate_dataset.py
```

3. Train and evaluate the model (prints accuracy, confusion matrix and report; saves confusion_matrix.png):

```
python phishing_detector.py
```

4. Check your own email — paste the text into `MY_EMAIL_TEXT` in the script, then:

```
python check_my_email.py
```

5. Merge a Kaggle dataset — save it as data/kaggle_dataset.csv, edit the column names in the script, then:

```
python merge_datasets.py
```

6. Scan an inbox — set the address and app password (preferably as environment variables), then:

```
python check_inbox.py
```